Logistic Regression: Gradient Descent and Code from Scratch

Contents

1. Overview . . . 2
2. The idea on one weight . . . 2
   - 2.1 A model with one weight . . . 2
   - 2.2 Where the slope comes from: one student . . . 4
3. Predictions in matrix form . . . 4
   - 3.1 One prediction . . . 4
   - 3.2 All predictions together . . . 5
4. The loss in matrix form . . . 6
5. The gradient . . . 8
   - 5.1 The first part . . . 8
   - 5.2 The second part . . . 9
   - 5.3 Putting them together . . . 10
6. The update rule . . . 12
7. The code . . . 14
8. Checking against scikit-learn . . . 15
9. Perfectly separable data . . . 16
10. Summary . . . 19
11. Sources . . . 19
12. Key terms . . . 20

Where this fits: Pipeline map step 8 (Model). See the Course map.



- **Builds on:** Logistic regression.
- **Compare with:** Support vector machines.

1. Overview

Key point: Gradient descent (G-862) walks the weights downhill on the loss. For logistic regression each student pulls the weights by the size of the student's prediction error, and the average pull is the step. The code built on this reaches the same answer as scikit-learn.

The loss function gave logistic regression its log loss, which has no closed-form minimum (no formula that solves for the lowest point). This Note finishes the journey:

1. watch gradient descent work on a model with a single weight;
2. write the predictions and the loss in matrix form;
3. differentiate the loss with respect to the **weights** (G-407; the coefficients, the numbers the model learns);
4. code gradient descent and check it against scikit-learn.

2. The idea on one weight

Key point: Start with any weight. Measure the slope of the loss there. Step downhill by learning rate × slope. Far from the minimum the slope is large and the steps are big; close to it the slope is small and the steps are tiny.

Gradient descent (G-862) was built step by step in the idea of gradient descent. The recap in one line: the slope of the loss curve tells which way is downhill and how far away the bottom is, and each step moves the weight by the **learning rate** (G-1068) times that slope.

2.1 A model with one weight

To see the steps, take the smallest possible logistic regression: one **feature** (G-772) (an input variable), one weight w, no **intercept** (G-960). The data is 30 students. The feature is the CGPA shifted down by 6:

$$x = \text{CGPA} - 6$$

The **target** (G-1949) (the output we predict) is $y = 1$ for placed and 0 for not placed. The model is

$$\hat{y} = \sigma(wx)$$

where $\hat{y}$ is the predicted probability of being placed and σ is the **sigmoid** (a curve that turns any number into a probability between 0 and 1, see the sigmoid function).

The shift by 6 is needed because this model has no intercept. Whatever w is, the curve gives probability 0.5 at $x = 0$:

$$\sigma(w \times 0) = \sigma(0) = 0.5$$

So $x = 0$ has to sit where placed and not-placed students meet, near CGPA 6 (the fitted curve with an intercept crosses 0.5 at CGPA 6.01, see one feature: an S-curve).

Figure 1 runs gradient descent on this model with **learning rate** (G-1068) 4, starting from $w = 0$. Watch the red ball on the left: its jumps shrink as the curve flattens. On the right, the S-curve of the current w steepens until it fits the students.

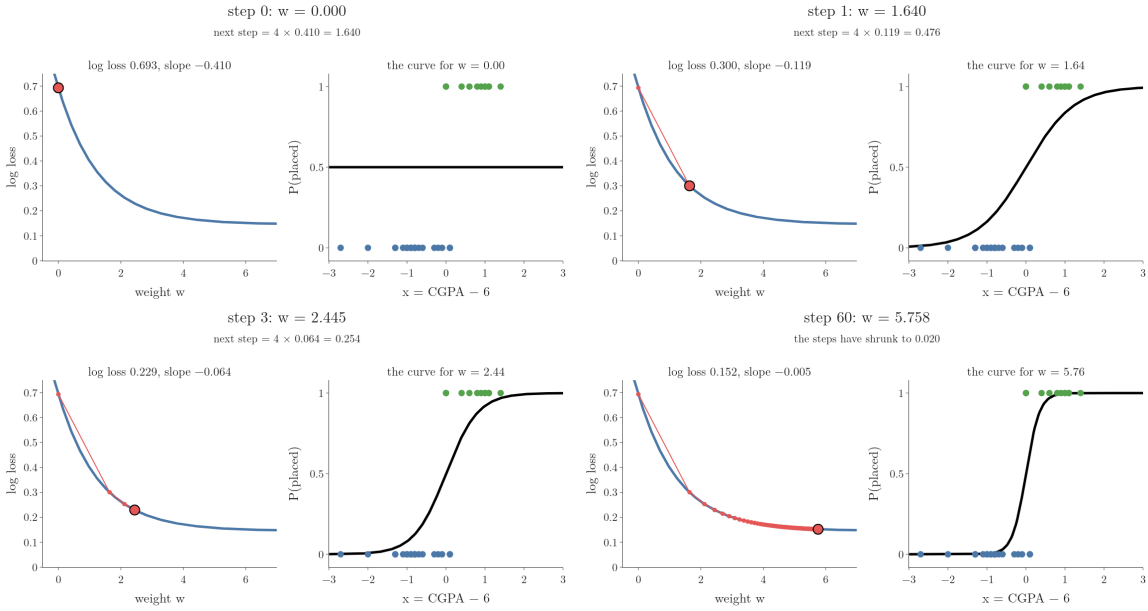


Figure 1: Gradient descent on one weight. Left: the log loss against w, with the ball at the current weight; each step is the learning rate (4) times the slope under the ball. Right: the curve $\sigma(wx)$ for the current w over the 30 students (green: placed, blue: not placed). The steps shrink from 1.640 to 0.020 over 60 steps while the log loss falls from 0.693 to 0.152. The picture of a ball taking big steps far from the minimum and small steps near it follows StatQuest, "Gradient Descent, Step-by-Step"; the data and model are our own

Step	w before	Log loss	Slope of the loss	Step $= -4\times$ slope	w after
1	0	0.693	-0.410	1.640	1.640
2	1.640	0.300	-0.119	0.476	2.116
3	2.116	0.253	-0.082	0.328	2.445
4	2.445	0.229	-0.064	0.254	2.699

The slope is negative, so the loss falls as w grows, and every step moves w to the right. After 60 steps the slope is -0.005 and the step is down to 0.020.

2.2 Where the slope comes from: one student

The slope in the table is an average over the 30 students. One student shows what goes into it. Take a student with CGPA 6.8 who was placed: $x = 0.8$, $y = 1$. At $w = 0$:

1. **Predict.**
$$\hat{y} = \sigma(0 \times 0.8) = \sigma(0) = 0.5$$
2. **Error.** The prediction is too low:
$$y - \hat{y} = 1 - 0.5 = 0.5$$
3. **This student's slope.**
$$-(y - \hat{y})\, x = -0.5 \times 0.8 = -0.4$$
4. **This student's vote for the step.** With **learning rate** (G-1068) 4:
$$-4 \times (-0.4) = +1.6$$

   The vote is to raise w, which raises $\hat{y}$ for this student.

Every student gives such a slope, $-(y_i - \hat{y}_i)x_i$, where the small i numbers the students: $i = 1$ is the first student, $i = 2$ the second. Their average at $w = 0$ is the -0.410 of the table. Sections 3 to 5 show where the expression $-(y - \hat{y})x$ comes from and write it for any number of **features** (G-772) at once.

3. Predictions in matrix form

> **Key point:** All m predictions at once: $\hat{y} = \sigma(\mathrm{Xw})$, where X has a first column of 1s.

3.1 One prediction

> **Key point:** For one observation, $\hat{y} = \sigma(\mathrm{w}_0 + \mathrm{w}_1\mathrm{x}_1 + \ldots + \mathrm{w}_n\mathrm{x}_n)$.

Take a dataset with m **observations** (G-1374) (records, one row of the data table each) and n features (one column each). The **target** (G-1949) y is the class, 0 or 1. The model has

$n+1$ weights: w_0 (the **intercept** (G-960), the part of the prediction that does not depend on any **feature** (G-772)) and w_1 to w_n. The prediction for observation i is

$$\hat{y}_i = \sigma(w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_n x_{in})$$

A tiny instance used through sections 3 to 5: $m = 3$ students and $n = 1$ feature, with x the CGPA shifted down by 6 as in section 2. The weights are $w_0 = 0.2$ and $w_1 = 1$.

Student i	x_{i1}	y_i	$z_i = 0.2 + 1 \times x_{i1}$	$\hat{y}_i = \sigma(z_i)$
1	0.8	1	$0.2 + 0.8 = 1.0$	0.731
2	-0.5	0	$0.2 - 0.5 = -0.3$	0.426
3	0.3	1	$0.2 + 0.3 = 0.5$	0.622

3.2 All predictions together

> **Key point:** Stack the rows: the inner sums are the matrix product Xw.

Writing out all m predictions, each inner sum is one row of X (with a 1 added in front) times the weight vector w. So all of them together are a single matrix product:

$$\hat{y} = \sigma(Xw)$$

where σ is applied to each entry. Stacking the predictions as Xw is the same trick as in multiple linear regression.

The notation on the tiny instance. X is the table of **features** (G-772) with a first column of 1s (one row per student, shape 3×2), and w is the weights written as one column:

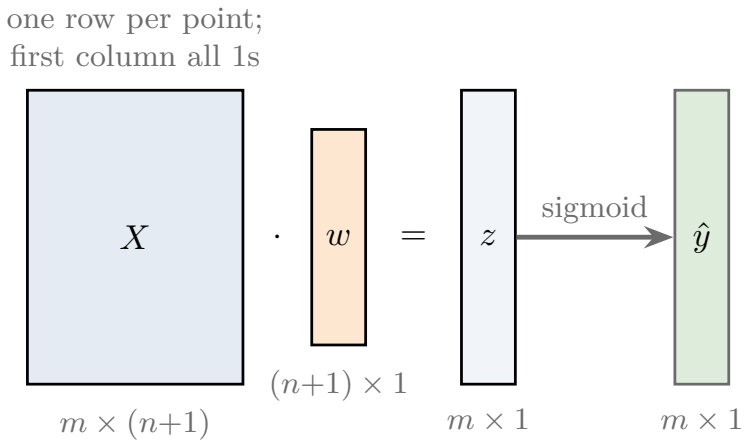
$$X = \begin{bmatrix} 1 & 0.8 \\ 1 & -0.5 \\ 1 & 0.3 \end{bmatrix}$$

$$w = \begin{bmatrix} 0.2 \\ 1 \end{bmatrix}$$

The product Xw takes each row of X, multiplies it entry by entry with w, and adds:

$$\text{row 1: } 1 \times 0.2 + 0.8 \times 1 = 1.0$$
$$\text{row 2: } 1 \times 0.2 + (-0.5) \times 1 = -0.3$$
$$\text{row 3: } 1 \times 0.2 + 0.3 \times 1 = 0.5$$

So $Xw = (1.0, -0.3, 0.5)$, the z_i column of the table above. Applying σ to each entry gives $\hat{y} = (0.731, 0.426, 0.622)$.



$$\frac{\partial L}{\partial w} = -\frac{1}{m} X^{\mathsf{T}}(y - \hat{y})$$

X^{T} is $(n{+}1) \times m$, $(y - \hat{y})$ is $m \times 1$: the gradient is $(n{+}1) \times 1$, one slope per weight, the same shape as w

Figure 2: The shapes of X, w, $\hat{y}$ and the gradient

In Figure 2, read the top row left to right: the m rows of X times the one column w give one z per **observation** (G-1374), and the sigmoid turns each z into a prediction.

4. The loss in matrix form

> **Key point:** $\mathrm{L} = -(1/\mathrm{m})[\mathrm{y}^{\mathsf{T}} \log \hat{y} + (1 - \mathrm{y})^{\mathsf{T}} \log(1 - \hat{y})]$.

First the loss on the tiny instance of section 3, one student per line (log is the natural logarithm). Each student has only one active term, because y_i is 0 or 1:

Student	y_i	$\hat{y}_i$	Active term	Loss of the student
1	1	0.731	$-\log 0.731$	0.313
2	0	0.426	$-\log(1 - 0.426)$	0.554
3	1	0.622	$-\log 0.622$	0.474

The loss L is the average of the three:

$$L = \frac{0.313 + 0.554 + 0.474}{3} = \frac{1.341}{3} = 0.447$$

The symbol $\sum$ (capital sigma, "sum") means "add up the terms for $i = 1, 2, \ldots, m$". Here, with $m = 3$:

$$\sum_{i=1}^{3}(\text{loss of student } i)$$

$$= 0.313 + 0.554 + 0.474 = 1.341$$

With this symbol, the log loss from the loss function is

$$L = -\frac{1}{m}\sum_{i=1}^{m}\Big[y_i \log \hat{y}_i$$

$$+(1 - y_i)\log(1 - \hat{y}_i)\Big]$$

The transpose y^{T} turns the column y into a row. The **dot product** (G-634) of a row and a column multiplies matching entries and adds them. With $y = (1, 0, 1)$ and the logs of the predictions:

$$\log \hat{y} = (\log 0.731, \log 0.426, \log 0.622)$$

$$\log \hat{y} = (-0.313, -0.854, -0.474)$$

The first dot product, one term per line:

$$1 \times (-0.313) = -0.313$$

$$0 \times (-0.854) = 0$$

$$1 \times (-0.474) = -0.474$$

$$y^{\mathsf{T}} \log \hat{y} = -0.787$$

This is the sum $\sum_i y_i \log \hat{y}_i$ written without the sign of summation. The second dot product, with $1 - y = (0, 1, 0)$ and the logs of the other probabilities:

$$\log(1 - \hat{y}) = (-1.313, -0.554, -0.973)$$

One term per line:

$$0 \times (-1.313) = 0$$

$$1 \times (-0.554) = -0.554$$

$$0 \times (-0.973) = 0$$

$$(1 - y)^\mathsf{T} \log(1 - \hat{y}) = -0.554$$

The loss is then:

$$L = -\frac{1}{3}(-0.787 - 0.554) = 0.447$$

The same loss as above. So, with the predictions

$$\hat{y} = \sigma(Xw)$$

the loss in matrix form is

$$L = -\frac{1}{m}\Big[y^\mathsf{T} \log \hat{y}$$

$$+(1 - y)^\mathsf{T} \log(1 - \hat{y})\Big]$$

5. The gradient

> **Key point:** Differentiate each part with the **chain rule** (G-371); the sigmoid derivative cancels the logs, leaving $-(1/\mathrm{m})\ \mathrm{X}^\mathsf{T}(\mathrm{y} - \hat{y})$.

5.1 The first part

> **Key point:** The derivative of y log $\hat{y}$ with respect to w is y$(1 - \hat{y})$ x.

For a single **observation** (G-1374), differentiate $y \log \hat{y}$ step by step with the **chain rule** (G-371) (multiply the derivatives of the nested steps):

- the derivative of $\log \hat{y}$ with respect to $\hat{y}$ is $1/\hat{y}$;
- the derivative of $\hat{y} = \sigma(z)$ with respect to z is $\hat{y}(1 - \hat{y})$ (see the derivation of the sigmoid derivative);
- the derivative of $z = w \cdot x$ with respect to w is x.

Here "derivative with respect to w" is written $\frac{\partial}{\partial w}$: the slope of the quantity when only w changes. Multiplying the three factors, on student 1 of the tiny instance ($y = 1$, $\hat{y} = 0.731$, $x = 0.8$):

$$y = 1$$
$$\frac{1}{\hat{y}} = \frac{1}{0.731} = 1.368$$
$$\hat{y}(1 - \hat{y}) = 0.731 \times 0.269 = 0.197$$
$$x = 0.8$$
$$1 \times 1.368 \times 0.197 \times 0.8 = 0.215$$

In symbols the $\hat{y}$ cancels, leaving $y(1 - \hat{y})\, x$:

$$y(1 - \hat{y})\, x = 1 \times 0.269 \times 0.8 = 0.215$$

The two lines give the same 0.215.

5.2 The second part

> **Key point:** The derivative of (1 − y) log(1 − $\hat{y}$) is −(1 − y) $\hat{y}$ x.

In the same way, for $(1 - y) \log(1 - \hat{y})$:

- the derivative of $\log(1 - \hat{y})$ with respect to $\hat{y}$ is $-1/(1 - \hat{y})$;
- the rest of the chain is the same: $\hat{y}(1 - \hat{y})\, x$.

Multiplying the factors, on student 2 ($y = 0$, $\hat{y} = 0.426$, $x = -0.5$):

$$1 - y = 1$$
$$\frac{-1}{1 - \hat{y}} = \frac{-1}{0.574} = -1.741$$
$$\hat{y}(1 - \hat{y}) = 0.426 \times 0.574 = 0.245$$
$$x = -0.5$$
$$1 \times (-1.741) \times 0.245 \times (-0.5) = 0.213$$

In symbols, $(1 - \hat{y})$ cancels, leaving $-(1 - y)\hat{y}\, x$:

$$-(1 - y)\hat{y}\, x = -1 \times 0.426 \times (-0.5) = 0.213$$

5.3 Putting them together

> **Key point:** $y(1-\hat{y}) - (1-y)\hat{y}$ simplifies to $y - \hat{y}$.

Add the two parts and multiply out the brackets:

$$y(1-\hat{y}) - (1-y)\hat{y}$$

$$= y - y\hat{y} - \hat{y} + y\hat{y}$$

The two $y\hat{y}$ terms cancel:

$$= y - \hat{y}$$

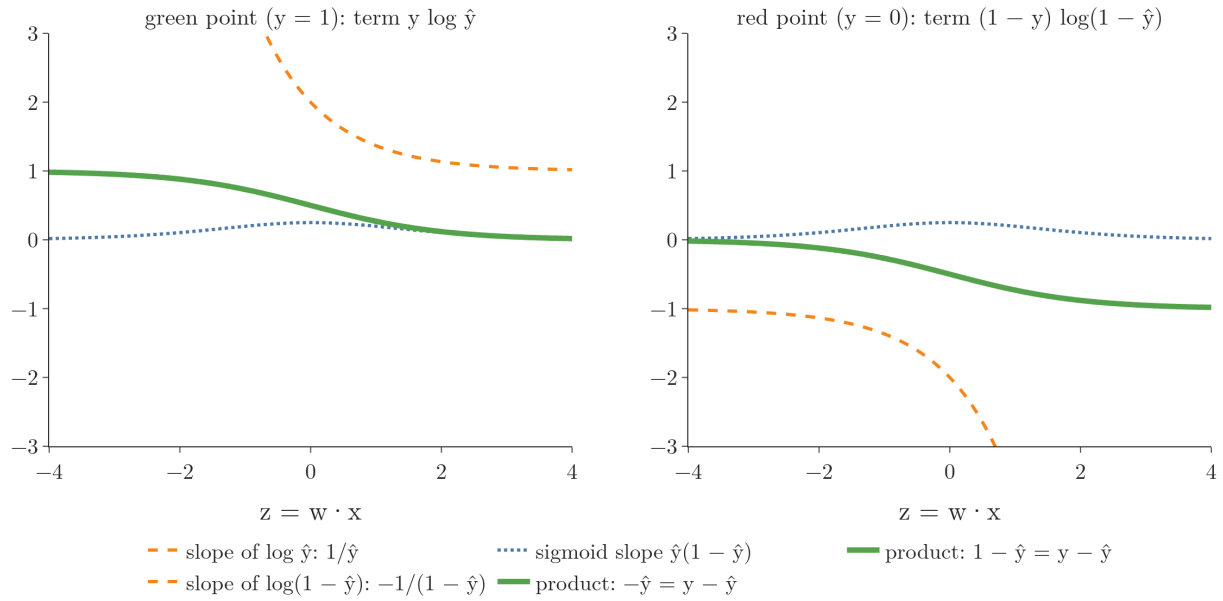


Figure 3: The cancellation along z. The log's slope (orange, dashed) blows up where $\hat{y}$ nears 0 or 1, and the sigmoid's slope (blue, dotted) shrinks there. Their product (green) is the plain $y - \hat{y}$.

In Figure 3, watch the green product stay between -1 and 1 even where the orange slope shoots off the chart: the two chain-rule factors cancel exactly.

So for one **observation** (G-1374), the derivative of the bracket is $(y - \hat{y})\,x$. On the tiny instance, the error $y - \hat{y}$ and its pull on the **feature** (G-772) weight w_1, student by student:

Student	$y - \hat{y}$	x_{i1}	$(y - \hat{y})\, x$
1	$1 - 0.731 = 0.269$	0.8	0.215
2	$0 - 0.426 = -0.426$	-0.5	0.213
3	$1 - 0.622 = 0.378$	0.3	0.113

Sum for w_1:

$$0.215 + 0.213 + 0.113 = 0.541$$

The same sum for the **intercept** (G-960) w_0 uses $x_{i0} = 1$:

$$0.269 - 0.426 + 0.378 = 0.221$$

The matrix product does both sums at once. The transpose X^T swaps the rows and columns of X:

$$X^\mathsf{T} = \begin{bmatrix} 1 & 1 & 1 \\ 0.8 & -0.5 & 0.3 \end{bmatrix}$$

$$y - \hat{y} = \begin{bmatrix} 0.269 \\ -0.426 \\ 0.378 \end{bmatrix}$$

Each row of X^T multiplies the column $y - \hat{y}$ entry by entry and adds.

Row 1, one product per line:

$$1 \times 0.269 = 0.269$$

$$1 \times (-0.426) = -0.426$$

$$1 \times 0.378 = 0.378$$

Row 2:

$$0.8 \times 0.269 = 0.215$$

$$(-0.5) \times (-0.426) = 0.213$$

$$0.3 \times 0.378 = 0.113$$

The two row sums make the result:

$$X^\mathsf{T}(y - \hat{y}) = \begin{bmatrix} 0.221 \\ 0.541 \end{bmatrix}$$

With the $-\frac{1}{m}$ in front ($m = 3$), the symbol $\frac{\partial L}{\partial w}$ (the slope of L for each weight, one number per weight) is:

$$\frac{\partial L}{\partial w} = -\frac{1}{m} X^\mathsf{T}(y - \hat{y})$$

$$= -\frac{1}{3} \begin{bmatrix} 0.221 \\ 0.541 \end{bmatrix}$$

$$= \begin{bmatrix} -0.074 \\ -0.180 \end{bmatrix}$$

Both slopes are negative, so **gradient descent** (G-862) raises both w_0 and w_1. In the general form, for any m and n:

$$\frac{\partial L}{\partial w} = -\frac{1}{m} X^\mathsf{T}(y - \hat{y})$$

X^T has shape $(n + 1) \times m$ and $(y - \hat{y})$ has shape $m \times 1$, so the **gradient** (G-863) has shape $(n + 1) \times 1$: one slope per weight, the same shape as w (Figure 2).

> **Extra:** This gradient has exactly the same form as the one for linear regression with mean squared error: X^T(actual − predicted), scaled. The only difference is that the predictions now pass through the sigmoid (Bishop §4.3.2).

6. The update rule

> **Key point:** Move each weight against its slope: w_new = w_old + η (1/m) X^T(y − $\hat{y}$).

Gradient descent (G-862) steps against the gradient, as in section 2. The step size η is the **learning rate** (G-1068). On the tiny instance with $\eta = 0.5$ and the **gradient** (G-863) $(-0.074, -0.180)$ of section 5.3:

$$w_0^{\text{new}} = 0.2 - 0.5 \times (-0.074) = 0.237$$
$$w_1^{\text{new}} = 1 - 0.5 \times (-0.180) = 1.090$$

In symbols:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

$$w_{\text{new}} = w_{\text{old}} + \frac{\eta}{m} X^{\mathsf{T}}(y - \hat{y})$$

Compare it with the sigmoid perceptron of the new update in action: $w \leftarrow w + \eta(y - \hat{y})x$ for one random point. The new rule is the same idea, averaged over **all** points in each step. The perceptron was one-point **stochastic gradient descent** (G-1892) on this very loss, run for a fixed number of steps. **Batch gradient descent** (G-264) uses all points in every step and, run to the end, reaches the true minimum.

	Sigmoid perceptron	Batch gradient descent
Update	$w \leftarrow w + \eta\,(y_i - \hat{y}_i)\,x_i$	$w \leftarrow w + \eta\,\frac{1}{m}\sum_i (y_i - \hat{y}_i)\,x_i$
Points used per update	one, picked at random	all m, averaged

Each point pulls the weights with a strength equal to its error $y_i - \hat{y}_i$. Figure 4 draws that strength as the size of each point on the data of section 8: 100 points with two **features** (G-772), in two classes that overlap a little.

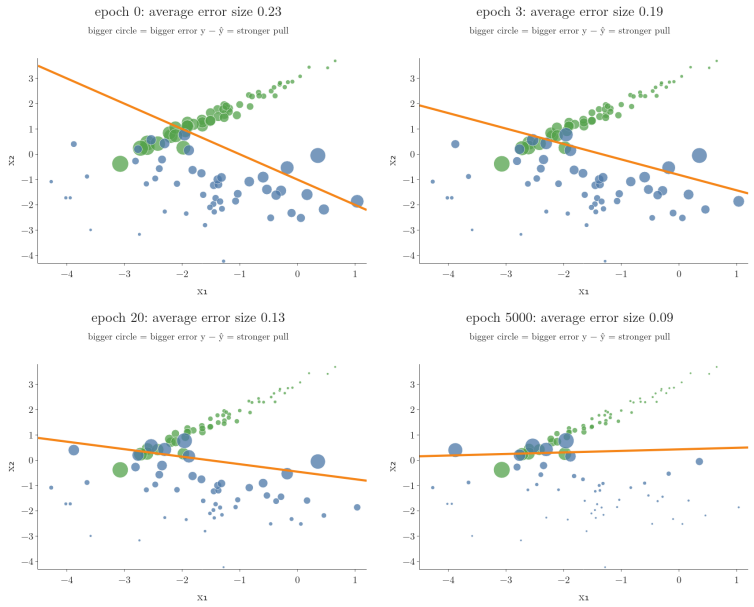


Figure 4: The 100 points of section 8 during batch gradient descent; the area of each circle shows the size of the point's error $y - \hat{y}$, and the orange line is the decision boundary. At the start many points have a large error and the boundary moves a lot per epoch. By epoch 5,000 only the points in the overlap still have a large error; their pulls balance out, and the boundary stops

1. **Epoch 0:** many circles are large (average error size 0.23). Their average pull is strong, so the boundary moves quickly.
2. **Epoch 20:** points far from the boundary on their correct side have shrunk to dots: a small error means a weak pull.
3. **Epoch 5,000:** the large circles left are the points in the overlap, on the wrong side or close to the boundary (average error size 0.09). The pulls from the two classes cancel in the average, so the update is close to zero.

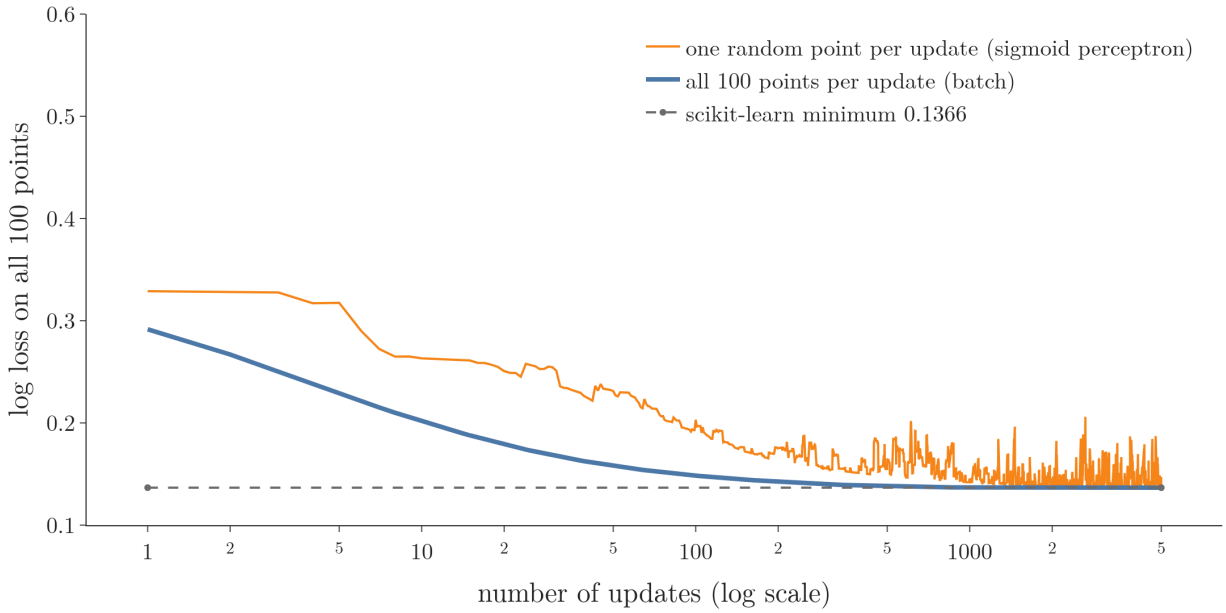


Figure 5: Log loss on the 100 points of section 8 against the number of updates, both starting from $w = (1, 1, 1)$. Orange: the sigmoid perceptron's rule, one random point per update (learning rate 0.1). Blue: the batch rule, all points per update (learning rate 0.5).

In Figure 5, watch the orange curve jitter above the dashed minimum (0.1383 after 5,000 updates) while the blue curve settles onto it (0.1366): each one-point step pulls towards a single point, the batch step towards all of them.

7. The code

Key point: Add a column of 1s, then repeat: predict with the sigmoid, and update with $\mathrm{X}^{\mathsf{T}}(\mathrm{y} - \hat{y})$ / m.

Python: Logistic regression with **batch gradient descent** (G-264).

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def gd(X, y, lr=0.5, epochs=5000):
    X = np.insert(X, 0, 1, axis=1)          # column of 1s
```

```
    w = np.ones(X.shape[1])
    for _ in range(epochs):
        y_hat = sigmoid(X @ w)                  # all predictions
        w = w + lr * (X.T @ (y - y_hat)) / X.shape[0]
    return w[1:], w[0]                          # coef_, intercept_
```

8. Checking against scikit-learn

> **Key point:** After 5,000 epochs, the weights match scikit-learn's to three decimals.

One **epoch** (G-696) is one pass over all the training points; **batch gradient descent** (G-264) makes one update per epoch.

The test data has 100 points with two **features** (G-772) whose classes overlap a little (`make_classification`, `class_sep=1.5`, random state 4). scikit-learn's `LogisticRegression(penalty=None)` fits the same model without **regularisation** (G-1659), so the two should agree.

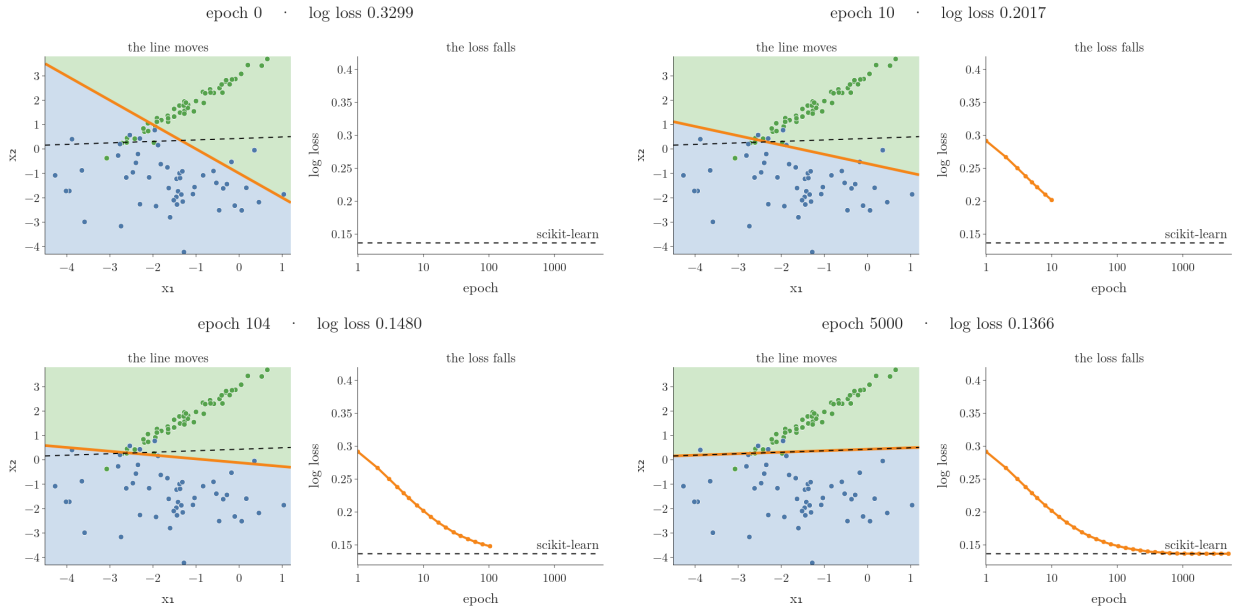


Figure 6: Batch gradient descent on the 100 points. Left: the decision boundary ($p = 0.5$) turns into place; the dashed line is scikit-learn's. Right: the log loss per epoch (log scale) falls to scikit-learn's minimum.

In Figure 6 the two pale colours are the two predicted classes (blue: $p < 0.5$, green: $p > 0.5$) and the orange line between them is the decision boundary, $p = 0.5$. Watch the orange line, the **decision boundary** (G-555) where $p = 0.5$, swing from its start at $w = (1, 1, 1)$ onto scikit-learn's dashed decision boundary while the loss curve flattens onto the dashed minimum.

	Intercept	w_1	w_2	Log loss
Start	1	1	1	
After 100 epochs	0.360	0.454	2.850	0.1484
After 1,000 epochs	−1.227	−0.094	3.542	0.1368
After 5,000 epochs	−1.568	−0.220	3.623	0.1366
scikit-learn	−1.569	−0.220	3.623	0.1366

The loss falls quickly at first and then levels off at scikit-learn's minimum (Figure 6, right). The decision boundary turns into place and ends on top of scikit-learn's (left). The model classifies 92% of the points correctly; the rest lie in the overlap, where no straight decision boundary can be perfect.

Python: The scikit-learn equivalent.

```
from sklearn.linear_model import LogisticRegression

lor = LogisticRegression(penalty=None)    # no regularisation
lor.fit(X, y)
lor.intercept_, lor.coef_                 # -1.569, [-0.220, 3.623]
```

Extra: By default, `LogisticRegression` adds an L2 (Ridge) penalty with `C=1`, so its weights are slightly smaller than these. `penalty=None` switches it off to match our code; the hyperparameters of logistic regression covers the options.

9. Perfectly separable data

Key point: If a straight **decision boundary** (G-555) separates the classes perfectly, the loss can always be made smaller by making the weights larger, so the weights never settle.

On the data of the perceptron Notes, where the classes are perfectly separated (`class_sep=20`), something different happens. This case is called **perfect separation** (G-1487):

Epochs	Weights (w_0, w_1, w_2)	Log loss
500	(3.63, 3.35, 0.11)	0.0052
5,000	(5.83, 4.84, 0.21)	0.00063
50,000	(8.20, 6.52, 0.37)	0.00007

The weights keep growing, and the loss keeps shrinking towards 0 without ever reaching it.

The next animation shades the plane by the model's probability, so first the surface behind it. The model's output for a point (x_1, x_2) is $p = \sigma(w_0 + w_1x_1 + w_2x_2)$ (the sigmoid ramp of the sigmoid function). With the weights after 50,000 **epochs** (G-696), $(8.20, 6.52, 0.37)$, the point $(0, 0)$ gets $p = \sigma(8.20) = 0.9997$. The surface is a steep step: flat near 0 on one side,

flat near 1 on the other, and a narrow ramp between. Figure 7 draws it. Seen from above it is a contour map: each line joins points with the same p (here 0.1, 0.5 and 0.9). Lines close together mean a steep ramp; the 0.5 line is the decision boundary, and the strip between the 0.1 and 0.9 lines is the "unsure band" of the next figure. Colours: green where p is high (class 1), blue where it is low (class 0).

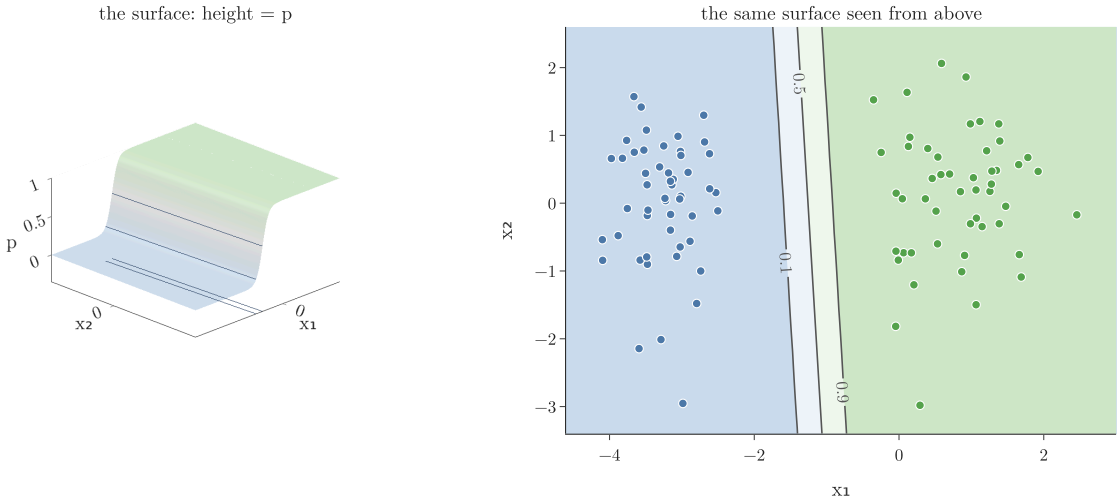


Figure 7: The probability surface of the model after 50,000 epochs on the perfectly separated points (left) and the same surface seen from above, a contour map with the lines $p = 0.1$, 0.5 and 0.9 and the data (right).

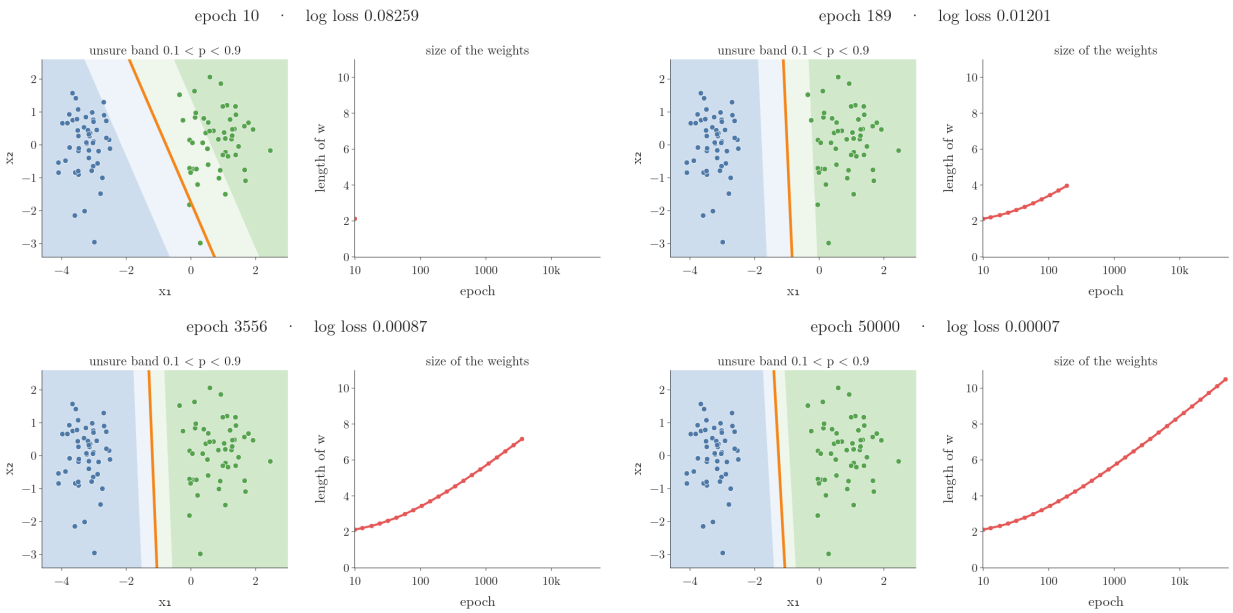


Figure 8: Batch gradient descent on perfectly separated points. Left: the pale band is where the model is unsure ($0.1 < p < 0.9$); the orange line is the decision boundary, $p = 0.5$. Right: the length of the weight vector against the epoch; the epoch axis is a log scale, each labelled tick ten times the one before.

In Figure 8, the decision boundary turns only slowly, yet the weights keep growing and the unsure band keeps narrowing: the model grows ever more confident instead of settling.

The reason: once every point is on its correct side, multiplying all the weights by 2 keeps the same decision boundary but makes every z twice as large, so every $\hat{y}$ moves closer to its correct 0 or 1. The loss always decreases, so there is no finite minimum (Bishop §4.3.2). The decision boundary itself still turns slowly: its slope $-w_1/w_2$ goes from -30 to -23 to -18 in the table. **Gradient descent** (G-862) turns it towards the decision boundary with the widest gap, but only very slowly (Soudry et al. 2018).

In practice this is handled by stopping after a fixed number of epochs, or by adding **regularisation** (G-1659), a penalty on large weights in the loss, the standard fix (Bishop §4.3.2). scikit-learn uses a penalty by default.

10. Summary

- Each step moves the weights by the **learning rate** (G-1068) times the slope of the loss: big steps far from the minimum, tiny steps near it, because the slope shrinks as the loss curve flattens.

Quantity	Formula	Why it matters
Predictions	$\hat{y} = \sigma(Xw)$	one matrix product gives all m predictions at once
Loss	$L = -\frac{1}{m}[y^\mathsf{T} \log \hat{y} + (1 - y)^\mathsf{T} \log(1 - \hat{y})]$	the dot products do the sum over the observations, so the code needs no loop
Gradient (thanks to $\sigma' = \sigma(1 - \sigma)$)	$\frac{\partial L}{\partial w} = -\frac{1}{m} X^\mathsf{T}(y - \hat{y})$	the sigmoid's slope cancels the log's, so each point pulls by its plain error $y - \hat{y}$
Update	$w \leftarrow w + \frac{\eta}{m} X^\mathsf{T}(y - \hat{y})$	the sigmoid perceptron's rule averaged over all points, so it settles on the true minimum instead of jittering

The code matches `LogisticRegression(penalty=None)`, so the derivation and the code are right: this is what scikit-learn computes.

- On perfectly separable data the weights grow without limit, because doubling them keeps the boundary and still lowers the loss; **regularisation** (G-1659) or early stopping keeps them finite.

11. Sources

Built from

- CampusX, "Logistic Regression Part 5 | Gradient Descent & Code From Scratch", YouTube, https://www.youtube.com/watch?v=ABrrSwMYWSg
- StatQuest with Josh Starmer, "Gradient Descent, Step-by-Step", YouTube, https://www.youtube.com/watch?v=sDv4f4s2SB8

Other references

- **Bishop:** Bishop, C. M. *Pattern Recognition and Machine Learning.* Springer, 2006. Section 4.3.2, pp. 205–207.
- **Soudry et al. 2018:** Soudry, D., Hoffer, E., Nacson, M. S., Gunasekar, S. and Srebro, N. "The Implicit Bias of Gradient Descent on Separable Data." *Journal of Machine Learning Research* 19(70), 1–57, 2018.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Perfect separation (G-1487)	When a line splits the training classes with no mistakes; unregularised weights then grow without limit.
penalty=None (G-1478)	The LogisticRegression setting that switches off the penalty on large weights (regularisation).
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Coefficient (β_i) (G-407)	The weight of one input column: the change in the output per unit of that input, others fixed.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Intercept (G-960)	The line's value when the input is 0; b in $y = mx + b$.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^\mathsf{T}x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
Gradient (of the loss) (G-863)	How fast the loss changes with each weight and bias, collected in one vector (the partial derivatives of the loss); it points in the direction in which the loss rises fastest.
Stochastic gradient descent (SGD) (G-1892)	Gradient descent that uses one random row for every update, so each update is cheap and many updates happen per epoch, at the cost of a noisier path.

Term	Meaning
Batch gradient descent (G-264)	Gradient descent that uses all training rows for every update.
Epoch (G-696)	One full update of the parameters using the whole training set.
Regularisation (G-1659)	Adding a penalty for large coefficients to the loss, so the model overfits less (lower variance).
Decision boundary (G-555)	A line or curve that separates the classes in classification.